Transform as the step of Prime's calculus: the verdict, with joined qualification

7 November 2026, revised

Abstract

We ask how far one guarded, information-carrying transformation of occurrences can serve as the unifying step of a calculus for higher-order MeTTa Prime, and where it cannot. The calculus is one rule over an abstract category of frames: a firing is admitted when its shapes hold, its selection is enabled, its information is solvable, and its guards and predicate queries hold at every solution; its result keeps the information along a settling morphism. The first-order, contextual and dependent steps are instances of that rule, and each feature is proved at the rule, or on a named restriction with a control that fails off it and a bridge back, or named as an open gap. The conflict-bearing first-order step is the η-long fibre of a dependent step with takes, reads, makes, tells, guards and predicate queries, and the admission of equation-guarded firings transfers through the fibre wherever the dependent unifier decides; both halves of "decides" are inhabited by actual kernel runs. Whether the unifier decides the whole dynamic pattern fragment is open: it is false for the earlier rule set, and the current solver's remaining gaps are named. The evidence forces revisions, each by a proved separation, and each stronger hypothesis is rejected with an obstruction under a stated adequacy criterion. No obstruction to the rule itself is known. We recommend retaining it as the operational core of the calculus, and list the open rows with their estimates.

Conventions. Lean declarations are cited by name. Unless they start with `Mettapedia`, names are relative to `Mettapedia.Languages.ProcessCalculi.ConstraintMultisetRewriting`. A rule's *patterns* say what a firing takes, reads and makes, what it tells the knowledge, and what it guards; a state's present occurrences form its *space*, and occurrences may carry a *handle* naming the space they belong to. Programs are written in MeTTa notation: variables `$x`, program equations `(= lhs rhs)` as schemas, typing `(: $x A)`; equations of the knowledge are written $\doteq$. Estimates of open rows are probabilities of success. All cited declarations use only the standard axioms `propext`, `Classical.choice` and `Quot.sound`, or fewer (Section 9).

Contents

1 The question and the answer **2**

2 The core calculus **3**

2.1 Syntax, frames and solutions 3
2.2 States 4
2.3 The rule 4
2.4 Identity of states 5
2.5 The fibre tower 5

3 One calculus: joined qualification **6**
3.1 The feature × instance table . . . 6
3.2 Features on a restriction . . . 7
3.3 The join through the fibre . . . 8
3.4 Other steps as instances . . . 9
3.5 Reconciliations and mismatches . . . 10

4 The requirements and their theorems **11**
4.1 Open unification . . . 12
4.2 Absence . . . 14
4.3 Composition . . . 15
4.4 Resources . . . 17
4.5 Evaluation and the common operations . . . 18
4.6 Reflection . . . 21
4.7 Inference . . . 22

5 Minimized examples **24**

6 Full programs from the core **26**

7 Recommendation **27**
7.1 Retain . . . 27
7.2 Revise . . . 27
7.3 Reject . . . 29
7.4 What stays open . . . 30

8 Addendum: results after the revision **31**
8.1 Firing of the pattern rules . . . 31
8.2 Causal networks on the step . . . 31
8.3 The C runtime and proof-system integration evidence . . . 32

9 Receipts **32**

1 The question and the answer

The question is whether one transform calculus coherently accounts for communication, querying and execution, together with ordinary Prime programs and checked inference, and if not, why not. The completion rule is strict: lists of results do not settle it, and an impossibility needs a proved obstruction under explicit adequacy criteria.

Answer. Retain the step. In detail:

1. *Retain.* Every behaviour in the seven decisive tests is decided by theorems about instances of one rule, and no obstruction to the rule is known (Section 7).
2. *One calculus.* The calculus is the one rule `Interface.Step` over the abstract category of frames `Interface.Frames`, with admission characterized once (`Interface.Step.admits_iff`). The first-order step, the contextual steps and the dependent full step are instances, and every other step the comparisons use is placed as an instance or a restriction of one, the contextual guarded firings at admission. Each feature is proved at the interface, or on a named restriction with a separating control and a bridge, or named as an open gap (Section 3).
3. *The join.* The second-order and first-order conflict-bearing steps are the η-long fibre of the dependent full step `Interface.DependentFull.step`. For firings whose guards and predicate queries are equations, admission transfers through the fibre wherever the dependent unifier decides the translated information (`Interface.DependentFull.fibre_admission_`

`transfer_of_full`); absence and counting transfer by separate theorems with their own hypotheses (Section 3.3). Both halves of "decides" are inhabited by actual kernel runs of the unifier: a solved translated pattern store whose firing is admitted on both sides, and a refuted one whose firing is refused on both sides (`Interface.DependentFull.NonVacuity.solved_case_nonvacuous`, `Interface.DependentFull.NonVacuity.refuted_case_refused`).

4. *Open.* Whether the dependent unifier decides every store of the dynamic pattern fragment is open. It is false for the earlier rule set (`ContextualDependentFragment.Controls.fragment_not_decisive`, `ContextualDependentFragment.Controls.blocked_not_decisive`). The current solver keeps every outcome of the previous one and decides both stores that refuted the earlier set; what is missing is firing on the flexible Miller-pattern equations, three rigid shapes, and closure of a stratified class under answers (Section 4.1). Everything stated "where the unifier decides" stays true whatever the answer.

The rule is not, by itself, the whole calculus, and the reasons are proved:

1. logical validity is not admission: inference is an *authority restriction* of the step's firings, and checked holes need a second, *closed* ask;
2. the frames must be dependent with η: the second-order frames are their η-long fibre, without η the translated pattern equations lose their principal answers, and principal answers transfer through the fibre in one direction only;
3. what the step may observe is split into ordinary queries, read through solutions, and reflection queries, which see syntax;
4. transport of native firings exists only up to dead names, with a move defined on part of the firings, and helper firings at dependent frames agree only up to the exchange of their telescopes;
5. concurrent waves need joint solvability of the whole wave;
6. atomic blocks of unbounded size need rules computed per block or a compound occurrence;
7. fresh identities are those never born, not those merely absent;
8. spaces share the knowledge, so a firing at one space can change admission at another through a shared unknown, and independence across spaces holds on the restriction that the firing adds no information; rights alone do not bound what a space receives;
9. contention between firings includes inhibition through predicate queries;
10. a guard that something bad is never reached has a finite certificate exactly on finite-state data, and is lowered to the check of one; reachability of a proof does not make fair search complete.

Each is a revision of the candidate as first presented, not an obstruction to the rule. The precisely stated stronger hypotheses are rejected in Section 7, each with its obstruction.

2 The core calculus

2.1 Syntax, frames and solutions

Terms are built over a binding signature: sorts, operators with binding arities, bound variables, and metavariables declared with arities $(\bar{s}, s)$ that are applied to terms of the sorts $\bar{s}$. A *frame* declares metavariables: the unknowns that exist and their sorts or types. The canonical frames are declaration families, the foundation's frames are formed dependent telescopes, and a substitution action has one frame.

Definition 1 (Frames acting on terms; `Interface.Frames`)**.** *A frame action has frames $a, b, \ldots$, terms T_a, morphisms $f : a \to b$ with an action $f_* : T_a \to T_b$, identities and composites, and an equivalence $\approx_a$ on each T_a, with* $(\mathrm{id})_* t = t$, $(g \circ f)_* t = g_* f_* t$ *and* $s \approx t \Rightarrow f_* s \approx f_* t$.

For an equation store P at a, a *solution* is a morphism $f : a \to b$ into any later frame with $f_* s \approx f_* t$ for every $(s, t) \in P$. $\mathsf{Solvable}(P)$ asks for one; $\mathsf{Entails}(P, \varphi)$ asks that $\varphi(b, f)$ hold at every solution; $u : a \to m$ is *principal* when it is a solution and every solution factors through

it (`Interface.Frames.Principal`), and then a store's solutions are exactly the factorizations through u (`Interface.Frames.Principal.models_iff`). Solvability is witnessed at a weakly terminal frame, and the frame with one parameterless metavariable per sort is weakly terminal, so existence never needs a higher-order helper (`Interface.Frames.solvable_iff_into`, `Interface.Frames.firstOrderFrame_weaklyTerminal`, `Interface.Frames.contextual_solvable_iff_consistent`). A substitution action is the one-frame case, whose solvability is consistency (`Interface.Frames.ofSubstitution_solvable_iff`).

Closed asks. Given a closed frame, a store is *closed-solvable* when it has a solution into it (`Interface.Frames.ClosedSolvable`). Symbolic and closed solvability agree under a principal answer whose frame has a closed point (`Interface.Frames.closedSolvable_iff_of_principal`).

2.2 States

A state $\langle \Gamma; K; W \rangle$ has a frame Γ, its *knowledge* K (an equation store at Γ) and its space W, a finite set of occurrence identities; each occurrence carries a label, a term at the frame. Occurrence identities are physical: equal labels at two identities are two occurrences. Fresh schematic variables of a rule are opened in a fresh scope at each firing; caller variables shared with the state are not renamed; the identities a firing makes are fresh.

2.3 The rule

Definition 2 (The step; `Interface.Step`)**.** *A firing f at a state $s = \langle \Gamma; K; W \rangle$ has a selection (the occurrences it* takes*,* reads *and* makes*), a target frame Γ' with an extension $e_f : \Gamma \to \Gamma'$ (its new unknowns), its* information *I_f at Γ', which contains $(e_f)_* K$ together with the equations matching its take and read patterns to the selected labels and the equations it* tells*, its shapes, and at every solution σ of I_f the predicate $H_f(\sigma)$: its* guards *and* predicate queries*. The one rule is*

$$\frac{\mathrm{Shapes}(f) \qquad \mathrm{Enabled}(W, f) \qquad \mathsf{Solvable}(I_f) \qquad \mathsf{Entails}(I_f, H_f)}{\langle \Gamma; K; W \rangle \xrightarrow{f} \langle \Gamma''; (c_f)_* I_f; (W \setminus \mathrm{taken}_f) \cup \mathrm{made}_f \rangle} \ [\textsc{Fire}]$$

where $c_f : \Gamma' \to \Gamma''$ is the firing's settling morphism.

The definition is exactly the fields of `Interface.Step`. Admission is characterized once, for every instance, by the field `Interface.Step.admits_iff`: the shapes, the enabling of the selection's commit at the present occurrences, semantic solvability of the information, and the ask of H_f at every solution of it. The result's space is $(W \setminus \mathrm{taken}) \cup \mathrm{made}$ (field `Interface.Step.present_result`), and its knowledge is the information carried along the settling morphism (fields `Interface.Step.settle` and `Interface.Step.knowledge_result`). The settling morphism is the identity on the dependent full step and an inclusion on the native step. Whether settling keeps solvable information solvable is a named property, `EvidenceInterface.SettlesConservatively`, not part of the rule.

Enabling requires the taken occurrences to be present, the read ones present and apart from the taken ones, and the made ones fresh. The shapes are each instance's own: the number of patterns, the distinctness of takes and makes, and the labels of what is made. Two read positions may select one occurrence.

The guard is asked at every solution of the firing's *own* information, which includes its own tells and its own matches: that is the *birth/guard cut*. A predicate query fixes the complete set of present occurrences in a region at every solution; absence is the empty case.

Runs and the cut. Runs are dependent words of admitted firings (`Interface.Step.Run`). Knowledge only refines along them: every solution of the last knowledge pulls back to a solution of the first (`Interface.Step.Run.models_start`). With unknowns read as proof obligations and solutions as proofs, a run's final solution is a proof of the start's obligations. What an admitted firing's guards and predicate queries answered at its cut holds at every later solution, read back to its target frame (`InterfaceTable.holds_of_admitted`, `InterfaceTable.cut_preserved`).

2.4 Identity of states

Three identities of states are used, each with its theorem.

- *Renaming of occurrences and equivalence modulo the knowledge* (`Observation.ModuloEquivalent`): a bisimulation of the ordinary step (`Observation.ModuloEquivalent.semantic_bisimilar`). Queries that read through solutions keep it; raw inspection of quoted syntax does not (`Program.rawQuoted_not_semantic`).
- *Up to dead names* on the native step: states that agree except on names no row, knowledge or ledger entry mentions are bisimilar (`Interface.NativeUpToDead.bisimilar_of_deadEquiv'`).
- *The races layer*: strong, weak and branching bisimilarity over any instance and any transport (`Race.Generic.Bisimilar`, `Race.Generic.WeakBisimilar`, `Race.Generic.BranchingBisimilar`); transport-equivalent states are bisimilar (`Race.Generic.bisimilar_of_same`). At dependent frames, firings that make helpers are alike up to an isomorphism of telescopes (`Interface.DependentFull.HelperAlike`).

2.5 The fibre tower

Level	Frames and the theorem that relates it to the next
one frame	a substitution action; consistency, entailment, principality (`Interface.Frames.ofSubstitution_solvable_iff`)
first order	parameterless metavariables of declaration families; weakly terminal, decides existence (`Interface.Frames.contextual_solvable_iff_firstOrder`); separately, first-order terms with their own unifier, which decides (`FirstOrder.accepted_iff_consistent`, `FirstOrder.accepted_principal`); no frame map between the two presentations is proved
patterns	metavariables applied to distinct bound variables; the driver solves or refutes (`ContextualPatternTermination.solves_or_refutes`); elsewhere residuals stay and lose no solution (`ContextualFrameWorklist.preserves_every_solution`, `BetaEtaReadings.FlexFlex.never_refuted`)
second order	declaration families; raised into dependent telescopes, with reflection of solutions and asks (`Interface.Translation.translation_reflects`)
dependent, with η	formed telescopes with typed $\beta\eta$; the second-order frames are its η-long fibre on translated frames (`ContextualDependentFibre.fibre_full`, `ContextualDependentFibre.fibre_solutions_correspond`); a dependent principal answer of a translated store at a translated assignment gives a second-order principal answer, one way only (`ContextualDependentFibre.fibre_principal_reflect`); a pattern equation has a principal answer (`ContextualDependentGeneralPattern.Pattern.replace_principal`); the driver returns principal answers when it solves, refutes only with a certificate, and never refutes a solvable store (`ContextualDependentWorklist.drive_solved_principal`, `ContextualDependentWorklist.drive_not_refuted_of_solvable`); decisiveness on the whole dynamic pattern fragment is open (Section 4.1)

The tower ends at the dependent frames with η because principality of pattern answers comes from η: over the dependent frames without η the translated pattern equation $m(x) \doteq n(x)$ has no principal answer (`Interface.Translation.pattern_no_principal`, `ContextualDependentFibre.Controls.point_pattern_no_principal`), and with η it has one (`ContextualDependentFibre.Controls.point_pattern_principal`, `Interface.Translation.EtaFrames.eta_pattern_principal`). Conservativity stops at frames the second-order side cannot express: at a frame with a Σ-typed variable a projection solves a translated store and is no translation (`ContextualDependentFibre.Controls.offImage_solution`). That a second-order principal answer stays principal against solutions into arbitrary dependent telescopes, with Π, Σ or identity types outside the image, is not proved; admission does not need it where the unifier decides, since the answers of the solvers in the tree land in translated telescopes (Section 3.3).

3 One calculus: joined qualification

The results we use come from several developments. They are about one calculus only if each step is an instance of Definition 2, each feature is proved at the interface or on a named restriction, and two definitions of one concept agree or are proved to differ.

3.1 The feature × instance table

The columns are the interface, the first-order step `AnnotatedRuns.firstOrder`, the contextual steps over declaration frames (the helper step `Interface.Helpers.helperStep`, with the native step `Interface.native` where the helper step lacks a feature), and the dependent full step `Interface.DependentFull.step`. In the three instance columns, unqualified names are in `InterfaceTable`. A *gap* is open.

Feature	Interface	First order	Contextual	Dependent full
frames and solutions	`Interface.Frames.Principal.models_iff`	`frames_firstOrder`	`frames_contextual`	`frames_dependent`
admission	`Interface.Step.admits_iff`	`admission_firstOrder`	`admission_helper`, `admission_native`	`admission_dependent`
runs	`Interface.Step.Run.models_start`	`runs_firstOrder`	`runs_helper`, `runs_native`	`runs_dependent`
the cut	`InterfaceTable.cut_preserved`	`cut_firstOrder`	`cut_helper`, `cut_native`	`cut_dependent`
guards	`InterfaceTable.holds_of_admitted`	`guards_firstOrder`	`guards_helper`	`guards_dependent`
predicate queries	absence: `Interface.DependentFull.entails_absent_iff`; count: `Interface.Frames.Principal.entails_count_iff`	`absence_firstOrder`, `count_firstOrder`	`absence_contextual`, `count_contextual` at frames; the helper step has none; the native step asks them (`admission_native`)	`absence_dependent`, `count_dependent`
spaces	`Spaces.run_executable`, `Spaces.present_result_spaces`, `Spaces.fiber_unchanged`; commuting only on a restriction	`SpacesInterface.located_independent`	`spaces_helper`, with no restriction	`SpacesInterface.dependent_independent`; gap: query regions are not located
permissions	`Permissions.refused_without_receive`, `PermissionsInterface.witness_present`; `PermissionsInterface.absence_preserved` on a restriction	`PermissionsInterface.old_owned_absence_preserved`	`witness_helper`; native absence: gap	`PermissionsInterface.dependent_absence_preserved`

Feature	Interface	First order	Contextual	Dependent full
disequations	`DisequalityFrames.frames`; `DisequalityFrames.Prestep.withDisequations`, `DisequalityFrames.Unguarded.withDisequations` on restrictions	`DisequalityFrames.oneFrame_admits_iff`	`diseq_helper`; `diseq_native` without guards; guarded native: gap	`diseq_dependent`
evidence rows	`Evidence.not_admits_of_not_solvable`; `EvidenceInterface.run_solvable` on a restriction	`EvidenceInterface.old_firstOrder_run_consistent`, `Evidence.Family.profiles_distinct`	`evidence_helper`, `evidence_native`; the four-way family: gap	`EvidenceInterface.dependent_run_solvable`, `EvidenceInterface.DependentFamily.profiles_distinct`
races and identity	`Race.Generic.bisimilar_of_same`	`races_firstOrder`	`races_helper`	`Interface.DependentFull.plain_bisimilar_of_same`, `Interface.DependentFull.helper_bisimilar_of_same`
programs	gap: one program core over the interface	the first-order core (Section 6)	the binder core; joined with the first-order core by agreement	gap

3.2 Features on a restriction

For each feature that does not hold on every instance, the restriction theorem, a control that fails off the restriction, and the bridge by which an instance meets it.

Feature	Restriction	Control off it	Bridge
spaces commute	*quiet*: the firing adds no information, its information being equivalent to the knowledge (`SpacesInterface.located_independent`, `SpacesInterface.dependent_independent`)	a shared unknown carries a binding across spaces (`Spaces.Controls.shared_variable_crosses_spaces`, `SpacesInterface.Controls.dependent_shared_variable_crosses_spaces`)	a syntactic check implies quiet (`SpacesInterface.quiet_of_exact`, `SpacesInterface.exact_independent`, `SpacesInterface.Controls.quiet_read_exact`)
absence under permissions	labels move along firings, `PermissionsInterface.LabelLaw` (`PermissionsInterface.absence_preserved`)	a firing that relabels a row spoils the absence (`PermissionsInterface.Controls.relabel_spoils_absence`)	`PermissionsInterface.locatedLabels_law`, `PermissionsInterface.dependentLabels_law`
disequations	a prestep whose settlement reflects equality (`DisequalityFrames.Prestep.withDisequations`)	a step that settles by guessing passes every admission test and holds $0 \neq 0$ (`DisequalityFrames.Restriction.guess_breaks_frame`, `DisequalityFrames.Restriction.guess_not_reflects`)	`DisequalityFrames.firstOrder_reflects`, `DisequalityFrames.Dependent.reflects`; for steps whose result needs its own admission, firings without guards (`DisequalityFrames.Unguarded.DFiring.admits_base`, `DisequalityFrames.Unguarded.native_unguarded`)

Feature	Restriction	Control off it	Bridge
no inconsistent store	`EvidenceInterface.SettlesConservatively` (`EvidenceInterface.run_solvable`); a retraction suffices (`EvidenceInterface.conservative_of_retracts`)	a step that settles by guessing enters $0 \doteq 1$ (`EvidenceInterface.Controls.guess_enters_inconsistency`, `EvidenceInterface.Controls.guess_not_retracts`)	`EvidenceInterface.firstOrder_retracts`, `EvidenceInterface.helper_retracts`, `EvidenceInterface.dependent_retracts`, `EvidenceInterface.withDisequations_retracts`; native: `EvidenceInterface.Native.native_conservative`

There is no single generic theorem for commuting spaces: a locality hypothesis general enough to state one would restate the residual law and be met by one instance only, so independence is proved per instance into the interface's own notion of independence.

3.3 The join through the fibre

The dependent full step. `Interface.DependentFull.step` is an instance over the typed $\beta\eta$ frames. A firing makes a telescope of fresh helpers, has take and read patterns and made rows at the extended frame, tells and guards equations of typed terms, and asks predicate queries that read through solutions; absence and counting are instances. Its admission is the interface's (`Interface.DependentFull.admits_iff`); its takes really constrain (`Interface.DependentFull.Firing.admitted_take_unifies`); after a firing every row is typed, a made row at its checked type and every other row as the old label carried along the weakening (`Interface.DependentFull.Firing.subject_reduction`). Told equations between different types are heterogeneous, and are refused only through solvability (`Interface.DependentFull.Controls.tell_heterogeneous`); an ill-typed make is refused (`Interface.DependentFull.Controls.illTyped_refused`).

Admission transfer. For candidate steps over the second-order frames whose guards and predicate queries are equations, admission equals the dependent admission of the translated data whenever the unifier decides the translated information inside the fibre (`Interface.DependentFull.fibre_admission_transfer`), under the package's standard facts, a closed realization and injective names. That condition is derived, not assumed: the full solver deciding the translated store at some budget gives it (`Interface.DependentFull.decidedInFibre_of_full`, `Interface.DependentFull.fibre_admission_transfer_of_full`). Absence and counting go through separate theorems with their own hypotheses. For the first-order step at the second-order endomorphism action, admission without census queries is translated admission when the unifier decides the problem and every absence extension by an endomorphic answer or a refutation (`Interface.DependentFull.admits_translate_iff`), with equal results (`Interface.DependentFull.result_translate`); counting checks transfer under the same condition for each extension (`Interface.DependentFull.admits_translate_iff_count`). Census queries with arbitrary regions are not translated: a second-order region has no meaning at a dependent solution outside the image. A second-order step over several frames with equation guards needs no endomorphism condition (`Interface.DependentFull.translateEquation_admits_iff_of_full`).

Solvers that keep translated telescopes. Every rule of the pattern and full solvers keeps the frame, removes the newest metavariable, or replaces two metavariables by a pruned one, and each keeps translated telescopes (`Interface.DependentFull.full_keeps_translated`). The later solvers do too: rotation only permutes chains of sorts, and preparation normalizes what is already normal under the checkable condition that the root evaluator is silent on chains of

sorts, which the evaluator without root steps meets (`Interface.DependentFull.normalizing_keeps_translated`, `Interface.DependentFull.permuting_keeps_translated`, `Interface.DependentFull.fragment_keeps_translated`, `Interface.DependentFull.identity_keeps_translated`, `Interface.DependentFull.pi_keeps_translated`, `Interface.DependentFull.chainsSilent_none`, `Interface.DependentFull.decidedInFibre_of_keeps`).

Non-vacuity. In the package with one sort constant, the actual full solver, evaluated by the kernel, solves the translated $m(x) \doteq n(x)$ with a principal answer at a translated telescope of one metavariable and refutes $F(x) \doteq y$ (`Interface.DependentFull.NonVacuity.pattern_commit`, `Interface.DependentFull.NonVacuity.take_transfer`). A second-order firing whose information is the first store is admitted on both sides (`Interface.DependentFull.NonVacuity.solved_case_nonvacuous`); the firing for the second is refused on both sides, with an independent second-order proof (`Interface.DependentFull.NonVacuity.refuted_case_refused`, `Interface.DependentFull.NonVacuity.outside_unsolvable_direct`). Off the image, a firing with a Σ-typed helper is admitted and its result is outside the fibre, without disturbing admission on it (`Interface.DependentFull.FibreControls.pair_admitted`, `Interface.DependentFull.FibreControls.pair_result_not_translated`).

Concurrency at dependent frames. Two helper-free firings are independent when they take disjoint occurrences, neither takes what the other reads, they make distinct occurrences, their joint information is solvable, and neither writes into the other's query regions; then both orders are admitted and the composites have the same occurrences and labels, with equivalent knowledge (`Interface.DependentFull.local_diamond`), and every clause but the query clause is necessary (`Interface.DependentFull.clauses_of_residuals`). One-helper firings close up to the swap of their helpers (`Interface.DependentFull.helper_diamond`), a helper firing and a helper-free one close on the nose (`Interface.DependentFull.mixed_diamond`), and firings that make whole telescopes close up to the block exchange, the unique map between the two pullbacks (`Interface.DependentFull.telescope_diamond`, `Interface.DependentFull.exchange_unique`). Without the exchange the composites differ (`Interface.DependentFull.TelescopeControls.composites_differ`); a made row whose type mentions the other firing's helper is the residual of no firing (`Interface.DependentFull.no_residual_of_mentions`). Helper firings are placed in the races layer up to an isomorphism of telescopes (`Interface.DependentFull.helper_bisimilar_of_same`, `Interface.DependentFull.helper_branchingBisimilar_of_same`).

Open in the join. Census queries through the join (60%); the endomorphism condition of the first-order instance from the solver's answers, which needs inhabitants of dropped metavariables' types (45%); absence and counting through the multi-frame instance (65%); step bisimilarity for helper firings, which needs a strict transport (60%); the converse of dead-helper simulation, which needs strengthening of typed judgments (45%).

3.4 Other steps as instances

Step	Placing theorem
occurrence core	a core step is an admitted query-free firing of the first-order step, and core reachability is exactly a query-free run (`TransformQualification.core_step_iff_admitted`, `TransformQualification.coreReaches_iff_run`); with predicate queries, `TransformQualification.query_step_iff_admitted`. This places the resource, census, snapshot, Need-machine and register-comparison results, each over its own substitution action

Step	Placing theorem
compiled clauses	a compiled clause is admitted exactly when its constructed firing is admitted by the native step (`TransformQualification.Native.admitted_iff_native`)
contextual guarded firings	a guarded firing at an execution endpoint is admitted exactly when its query-free native firing is, at every budget (`TransformQualification.Native.guarded_valid_iff_native`); this covers the contextual λ, pair, causal and schematic-storage executions at admission. Their results are not compared with the native published state
live native step	`Interface.NativeUpToDead.live`, the native step on firings with pattern queries, live outputs and no recipes
dependent and simple fibre	`Interface.Dependent.helperStep`; the simple-telescope fibre computes the full instance's admission when the knowledge has a principal answer at a simple frame (`Interface.Dependent.simpleStep_admits_iff`)
disequation steps	`Disequality.step`, an instance over the disequation frames of a substitution action (`Disequality.admits_iff_frames`); over any frames, `DisequalityFrames.Prestep.withDisequations`
ordinary step	`Semantic.semanticStep`, a restriction of the first-order step (`Semantic.semanticStep_admits_iff`)
program core	translated runs are runs of the ordinary step, exactly (`Program.program_interface_iff`); the scope of fresh rule variables is threaded by the translated run, not by the frame extension
binder program core	each translated equation firing is a step of the contextual guarded step (`ProgramBinders.destination_is_core_step`), and the unmatched return is a guarded firing (`ProgramBinders.unmatched_is_guarded_step`)
typing	`TypingChecking.goalStep`, `TypingStates.computeStep`, `TypingCheckingMeta.metaGoalStep`: instances by construction; call contracts are first-order firings (`TypingFirings.closed_call_admits_iff`)
inference hosts	a step of a schema process is exactly an admitted query-free first-order firing whose rule is an installed schema's activation and whose made occurrences the process may publish to (`TransformQualification.schema_step_iff_admitted`, `TransformQualification.rule_step_iff_admitted`)
restrictions by rights	any instance restricted by a predicate on firings is again an instance (`Spaces.restrict_admits_iff`)
MM2 `exec`	a covered `exec` over a duplicate-free ground representation is one firing (`MeTTaOperationsExec.exec_is_one_firing`)
outside	the abstract machines of the Need lane (for instance `Mettapedia.Machines.BranchLocalNeed.CallTimeChoiceInstructions.source_iff_executable`) and the register semantics of Meta-MeTTa are comparison sources, not instances

3.5 Reconciliations and mismatches

Concept	Two definitions, and the theorem relating them
a step of occurrences	the core relation and the first-order instance coincide (`TransformQualification.core_step_iff_admitted`)

Concept	Two definitions, and the theorem relating them
absence	the guard absent(p) and the predicate query with an empty census for the region of p hold at the same solutions, and moving the guard into the queries keeps admission (`TransformQualification.absenceQuery_holds_iff`, `TransformQualification.admits_absent_check_iff_query`); the two program hosts' absence checks agree on embedded first-order rows, and so do their return firings (`ProgramHosts.absence_agrees`, `ProgramHosts.return_admitted_iff`)
closed solvability	the interface's closed solvability and the dependent unifier's are the same at the typed $\beta\eta$ frames, definitionally (`TransformQualification.Closed.closedSolvable_iff_frames`)
transport	strict transports are transports up to dead names with the same state classes (`Interface.Transport.toDead_same_iff`); the strict separation law fails natively (`Interface.NativeUpToDeadLaws.strict_separation_fails`)
queries	a raw label query is not invariant under knowledge equivalence (`Semantic.SemanticControls.syntactic_collision`); a query read through solutions is (`Semantic.SemanticControls.noYSemantic_agrees`); the same split at dependent frames (`Interface.DependentFull.Controls.syntactic_collision`)
freshness	absent from the space versus never born: a firing whose made identity is absent and was born earlier is admitted and is realized by no commit protocol run after that history (`Interface.NativeFreshnessControls.maker_absent`, `Interface.NativeFreshnessControls.maker_not_realized`); the protocol realizes exactly the runs fresh from the born identities (`Interface.native_run_protocol_iff`); a space is born by a firing that makes its marker, and "fresh" is never born (`SpacesCreation.created_never_born`)
solvability	symbolic and closed solvability differ at an obligation of an empty type (`Interface.Dependent.obligation_trap`, `ContextualDependentClosedBoundary.Controls.point_not_closedSolvable`) and agree under a principal answer with a closed point (`ContextualDependentClosedBoundary.principal_closed_iff`)
independence	the interface's semantic diamond and the static independence of guard-free firings coincide on static firings (`AnnotatedRuns.firstOrder_independent_iff_static`)
first-order terms	the one-frame instance over first-order terms and the parameterless fibre of the declaration families are two presentations; the occurrence core is generic over substitution actions and is placed for both, but no map between the two is proved
application	the closure machine of the program core computes substitution β, both ways, on closed terms (`ProgramSubstitution.plotkin_closed`, `ProgramSubstitution.machine_iff`), and every such evaluation is a run of the contextual λ execution (`ProgramSubstitution.eval_executes`); β as an equation of the binder core agrees with both on runs (`ProgramBinders.Nested.lambda_three_way`)
compiled and interpreted programs	one compiled firing answers a call exactly when the fixed interpreter's run answers it, up to renaming both ways, for calls and rows below the call's scope and rows that are data (`ProgramInterpreterJoin.interpreter_join_iff`)

4 The requirements and their theorems

Each row gives the requirement, the theorems, the instance, and a classification: a *restriction* of the step, a *compositional encoding* over it, a *necessary extension*, or a *proved obstruction*. The status is proved unless a row says otherwise. The separating example of each row is in Section 5 or named in the row.

4.1 Open unification

Requirement	Theorems	Instance; class
compatible shared constraints	two interactions with shared equations commute when resources are independent, the joint equations are consistent and writes avoid the other's guards (`Core.independently_ready_commute`, `Core.Controls.nontrivial_guarded_interactions_commute`); the tells $X \doteq 1$, $Y \doteq 2$ are independent (`AnnotatedRuns.LawfulControls.unlawful_changes_independence`)	first order; restriction
incompatible shared constraints	tells of $X \doteq$ true and $X \doteq$ false: each alone, neither after the other (`Interface.NativeResidualExample.tellers_not_after`); at dependent frames, $X \doteq$ Ground against $X \doteq$ Ground $\to$ Ground (`Interface.DependentFull.IndependenceControls.tells_clash_refused`)	native; dependent full
fresh local variables	helper families renamed by isomorphisms keep solvability (`Interface.Frames.Iso.solvable_applyStore_iff`); consumption and production need the fresh family (`ContextualExecutionControls.HelperConsumption.actual_consumption_and_production_require_the_fresh_family`); helper telescopes at dependent frames (`Interface.DependentFull.telescope_diamond`). The rows on the one-frame first-order step (`TransformQualification.Examples.OpenUnification.fresh_second_read_admitted`, the program core) exercise freshness by scope numbering, not by frame extension	contextual; dependent full; encoding
genuine binder crossing	composition under a real binder (`ContextualUnifierComposition.Controls.both_orders_act_under_a_real_binder`); dependent pattern answers (`ContextualDependentGeneralPattern.Pattern.replace_principal`); principal with η only (`ContextualDependentFibre.Controls.point_pattern_principal` against `ContextualDependentFibre.Controls.point_pattern_no_principal`); a take through a binder, admitted on both sides of the fibre (`Interface.DependentFull.FibreControls.binder_take_admitted`)	dependent η frames; extension of the frames
intentional caller sharing	a shared second read is refused (`ContextualSchematicStorage.Controls.the_explicit_shared_second_read_is_refused`, on the activated step, whose shared-row, guard-free profile is the occurrence core over the contextual action, `ContextualActiveStepProfile.step_iff`; that this control lies in the profile is not proved); a shared output keeps one reading, a reusable one has independent readings (`ContextualPublication.Controls.shared_publication_keeps_one_shared_reading`, `ContextualPublication.Controls.the_reusable_output_has_both_independent_readings`)	activated step; restriction

Requirement	**Theorems**	**Instance; class**
solver guarantees	first order decides; patterns decide; the dependent driver never refutes a solvable store and loses no solution (`ContextualDependentWorklist.drive_no_loss`); decisiveness on one-equation stores where a rule fires (`ContextualDependentDecisiveFragment.single_decided`); a non-pattern store is suspended by the fragment and rigid solvers at every budget and number of rounds (`ContextualDependentFragment.Controls.repeated_pending`)	tower
the earlier rule set does not decide the fragment	two fragment stores are decided at no budget: the unsolvable $\lambda a\, b.\, \mathrm{Id}\, T\, a\, b \doteq \lambda a\, b.\, \mathrm{Id}\, T\, b\, a$ and the solvable $T \doteq A$ over $A, T, G : (x{:}A) \to T$ (`ContextualDependentFragment.Controls.fragment_not_decisive`, `ContextualDependentFragment.Controls.blocked_not_decisive`)	proved obstruction for that rule set
repairs	identity and Π decomposition (`ContextualDependentFragment.Controls.id_identity_decided`, `ContextualDependentFragment.Controls.pi_principal`); solving in place (`ContextualDependentInPlace.Controls.blocked_decided_principal`); the occurs check along domains, codomains and carriers (`ContextualDependentOccursFull.occurrence_not_typeEq`, `ContextualDependentOccursFull.Controls.codomain_in_fragment_refuted`); a rank that every decomposition lowers (`ContextualDependentRank.weightRank_wf`, `ContextualDependentRank.rankedSettledOfConst`); rigid clashes under abstractions (`ContextualDependentFragmentDecisive.ClashControls.clashes_decided`)	dependent η frames
the census and the stratified class	each fragment equation is classified by the shapes of its opened sides (`ContextualDependentCensus.census_fragment`); a class beyond the fragment whose annotations agree under every solution of the earlier equations contains it and is kept by the scan and the twin step (`ContextualDependentClosure.stratified_of_fragment`, `ContextualDependentClosure.Stratified.splice`, `ContextualDependentAcceptance.scan_stratified`, `ContextualDependentAcceptance.collapse_stratified`); the solver chain keeps every outcome of the previous family (`ContextualDependentAcceptance.drive_conservative`, `ContextualDependentAcceptance.arguments_decided_of_any`)	dependent η frames
decisiveness on the whole dynamic pattern fragment	*open*, 45%. Missing: progress of the pattern, in-place and prune rules on every Miller-pattern equation (40%); three rigid shapes, each in the fragment and solvable, suspended at budget 16 (`ContextualDependentCensus.Controls.rows_unread`; nested and applied projections, spines at a flexible base type); closure of the stratified class under answers (45%)	tower

4.2 Absence

Requirement	Theorems	Instance; class
cancel(0) with u unknown	refused (`TransformQualification.AbsenceExample.ask_refused_unknown`)	first order; restriction
later $u \doteq 1$, $u \doteq 0$	admitted after $u \doteq 1$, refused after $u \doteq 0$; the run *bind* 1; *ask* (`TransformQualification.AbsenceExample.ask_admitted_after_one`, `TransformQualification.AbsenceExample.ask_refused_after_zero`, `TransformQualification.AbsenceExample.later_equation_run`)	first order
absence at every solution	absence holds at every solution exactly when each extension that would match is refuted, with no principal answer needed (`Interface.DependentFull.entails_absent_iff`); at dependent frames the answer can change through a type-level solution alone (`Interface.DependentFull.Controls.absence_refused_after_ground`, `Interface.DependentFull.Controls.absence_admitted_after_arrow`)	dependent full
contextual absence by certified refutation	"no equation matches" is one guard per equation on the guarded step, holding exactly when no equation matches any reading (`ProgramBinders.guarded_checks_iff`); decided by refuting a joint problem: sound (`ProgramBinders.certified_guarded`), complete for Miller patterns (`ProgramBinders.certifies_guarded_of_absent`), and never for a matched equation at any budget (`ProgramBinders.guarded_never_certified`)	contextual guarded step; restriction
absence stable under later bindings	every later instance of an unmatched call is unmatched (`ProgramNested.absence_stable`); under binders, in every frame (`ProgramBinders.absence_stable`)	ordinary; contextual
incomplete scans	a scan is sound exactly under coverage, and coverage is the apartness of every omitted row (`Check.entails_absent_iff_of_covered`, `Check.coversMatches_iff_omitted_apart`); the partial scan of the example lacks it (`FirstOrderChecks.Controls.OpenAbsence.the_partial_scan_lacks_coverage`); an unfinished search never reports none (`ContextualAttemptOutcomes.Attempt.unfinished_cannot_report_none`)	checks over the first-order step; restriction
relevant insertions	inserting *cancel*(1) after $u \doteq 1$ refuses (`TransformQualification.AbsenceExample.relevant_insertion_refuses`); in compiled clauses, `ContextualNativeGuardControls.a_matching_insertion_is_detected`	first order; native
unrelated updates	inserting *cancel*(2) keeps admission (`TransformQualification.AbsenceExample.unrelated_insertion_keeps`); writers outside a guard's region keep it (`Interface.Step.Guards.guard_fire`); `ContextualNativeGuardControls.an_unrelated_insertion_is_permitted`	first order; native

Requirement	Theorems	Instance; class
the original cut	the earlier cut refuses, a later one would accept (`ContextualNativeGuardControls.the_actual_earlier_query_is_refused`, `ContextualNativeGuardControls.choosing_the_later_cut_would_accept`); the firing's own equation counts (`ContextualNativeGuardControls.OwnEquations.the_clause_own_equation_is_retained`); an earlier count cannot be rebased (`ContextualCausalExecutionControls.the_original_count_cannot_be_rebased_to_the_final_cut`); at the interface, `InterfaceTable.cut_preserved`	native; interface
conflicting evidence against an inconsistent store	at knowledge with no solution no firing is admitted (`Evidence.not_admits_of_not_solvable`); rows for p and its opposition coexist with consistent knowledge and still steer admission (`Evidence.Rows.conflict_consistent`, `Evidence.Rows.accept_refused_with_opposition`); a store holding both tells admits nothing (`Evidence.Rows.both_tells_admit_nothing`)	first order; restriction
absence, refutation, incomplete search and disequality	four ways of not finding $p(a)$ give four distinct verdict profiles on four probes (`Evidence.Family.profiles_distinct`), and a scan that skips the open row would license the absence (`Evidence.Family.incomplete_scan_would_license`); the same profiles on the dependent full step (`EvidenceInterface.DependentFamily.profiles_distinct`)	first order; dependent full
disequality tells	a necessary extension, built: disequation frames over any category of frames (`DisequalityFrames.frames`); after $X \neq a$, $X \doteq b$ is admitted and $X \doteq a$ refused (`Disequality.Controls.disequality_checked`); absence changes with an insertion, a disequality does not (`Disequality.Controls.absence_vs_disequality`); conservative over equation-only knowledge (`Disequality.conservative`); principal answers live in constrained frames, and no single substitution presents $X \neq a$ (`Disequality.principal`, `Disequality.Controls.no_presenting_answer`)	disequation steps; necessary extension

4.3 Composition

Requirement	Theorems	Instance; class
resource independence with information interference	trace-independent footprints, and only one order runs, because a guard reads the other's tell (`AnnotatedRuns.Controls.told_check_one_order`); a query against a writer into its region (`AnnotatedRuns.Controls.census_writer_one_order`)	first order; restriction

Requirement	Theorems	Instance; class
retained guards	a later binding does not repair an earlier check (`ContextualScheduleControls.EarlierSchema.a_later_binding_does_not_repair_the_compiled_earlier_check`); a firing cannot discard a countermodel (`ContextualPrefixGuardControls.an_actual_firing_cannot_discard_that_countermodel`)	contextual; restriction
joint solutions against failure prefixes	every order of a jointly solvable clear family runs and all agree up to dead names (`Interface.NativeIterated.every_order_admitted`, `Interface.NativeIterated.every_order_agrees`); without joint solvability no order runs (`Interface.NativeIterated.no_order_of_not_jointly`); a refused joint problem keeps the executed producer prefix (`ContextualCausalExecutionControls.joint_refusal_does_not_erase_the_executed_producer_prefix`); a refused block leaves no residue (`StagedSourceEffects.refused_has_no_residue`)	native; first order
independence is a property of the step	two lawful transports with the same state classes have the same independent pairs (`Interface.Transport.Lawful.independent_iff`); an unlawful residual changes independence (`AnnotatedRuns.LawfulControls.unlawful_changes_independence`)	interface
congruence	rule-labelled bisimilarity is a congruence for disjoint contexts (`Semantic.ruleBisimilar_tensor`); closed bisimilarity of a program is not (`Observation.CongruenceControls.closed_not_congruence`)	first order
native diamonds	composites of a clear diamond are bisimilar and step bisimilar (`Interface.NativeDead.composites_bisimilar`, `Interface.NativeUpToDead.makers_composites_stepBisimilar`); a concurrent step is bisimilar to each sequentialization (`Interface.NativeConcurrent.Concurrent.bisimilar`)	live native
dependent diamonds	helper-free firings close on the nose under five independence clauses, each but the query clause necessary (`Interface.DependentFull.local_diamond`, `Interface.DependentFull.clauses_of_residuals`); helper firings up to the block exchange (`Interface.DependentFull.telescope_diamond`, `Interface.DependentFull.TelescopeControls.exchanged_diamond`, `Interface.DependentFull.TelescopeControls.composites_differ`); a dependency between the made rows excludes any exchange (`Interface.DependentFull.no_residual_of_mentions`)	dependent full
probabilities of causal histories	for positive rate systems and maximal runs of two parts, the jump-chain likelihood summed over all interleavings is the product of the parts' likelihoods (`CausalHistories.sum_shuffles`, `CausalHistories.history_factorizes`), with the ternary coherence (`CausalHistories.history_factorizes_three`); on two ground races of the step (`CausalHistories.StepControls.step_history_product`, `CausalHistories.StepControls.local_exact`); on a confused contention set the per-firing reading depends on the order (`CausalHistories.Controls.confusion_control`)	rate systems; first order

Requirement	Theorems	Instance; class
contention	contention is a shared take, a take of what the other reads, or inhibition: a take or make inside the other's predicate-query region, and contending firings are not independent (`Contention.not_independent_of_contends`); contention as sharing alone gives local likelihoods of total mass 2, contention as interference gives the step's (`Contention.Controls.inhibition_control`, `Contention.Controls.maker_guarded_dependent`)	first order with queries
interventions	a `do` restricts weights, composes by conjunction and commutes (`Intervention.restrictWeight_comm`), lowers exactly on static alphabets (`Intervention.prob_restrictWeight`), and at a space reaches only firings with the right to make there (`Intervention.doAt_of_no_send`); one atom through an absence guard performs a hard intervention (`Intervention.Controls.context_performs_do`)	first order; encoding

4.4 Resources

Requirement	Theorems	Instance; class
aliased reads	one occurrence read twice is admitted (`TransformQualification.Examples.Resources.aliased_read_admitted`); two read views count as one stored row (`SchematicReach.Controls.two_read_views_still_count_as_one_stored_row`); unrestricted aliasing admits a firing the register semantics has no counterpart for (`MetaMeTTaComparison.Family.unrestricted_read_aliasing_admits_another_firing`)	first order; restriction
distinct takes	one occurrence taken twice is refused (`TransformQualification.Examples.Resources.take_same_twice_refused`); a consumed cursor cannot be replayed (`NeedSnapshots.Controls.consumed_cursor_cannot_be_replayed`)	first order
occurrence multiplicity	two equal-valued occurrences are both taken (`TransformQualification.Examples.Resources.take_two_admitted`); equal payloads keep two cells (`NeedSnapshots.Controls.equal_payloads_keep_two_cells`)	first order
read before consume	reading then taking is a run, the converse order is refused (`TransformQualification.Examples.Resources.read_then_take`, `TransformQualification.Examples.Resources.read_after_take_refused`)	first order

Requirement	Theorems	Instance; class
several spaces	one space whose occurrences carry handles behaves as several: enabling and results split by handle, and the frame is unchanged (`Spaces.spaces_injective`, `Spaces.enabled_iff_spaces`, `Spaces.present_result_spaces`, `Spaces.fiber_unchanged`); every run of every instance is an executable commit word (`Spaces.run_executable`); on the located first-order step, admission at a handle depends only on its space and the knowledge (`Spaces.admits_iff_of_fiber`)	interface; located first order; encoding
independence across spaces	on the restriction *quiet* (Section 3); the knowledge is shared, so isolation between spaces is physical, not informational (`Spaces.Controls.shared_variable_crosses_spaces`)	first order, dependent full; restriction
creating a space	a firing that makes a birth marker at an unborn handle creates a space; markers are never taken, and every occurrence lies at a handle already born (`SpacesCreation.created_never_born`, `SpacesCreation.wellBorn_result`)	first order; restriction
rights per handle	a firing without `receive`, `peek` or `send` at a space is refused (`Permissions.refused_without_receive`, `Permissions.refused_without_peek`, `Permissions.refused_without_send`); without `receive` a space only grows and a witness survives, on every instance (`PermissionsInterface.witness_present`); without `send` it only shrinks and an absence survives, on instances whose labels move along firings (`PermissionsInterface.absence_preserved`)	interface; restriction
the authority escape	on runs of ticketed firings the rows made plus the tickets left never exceed the tickets at the start (`Permissions.made_le_tickets`); rights alone allow unbounded writes (`Permissions.Controls.rights_alone_unbounded`), a ticket per write blocks them (`Permissions.Controls.ticketed_blocks`)	first order; restriction
earlier space and permission layers	permitted runs are non-interfering (`Permissions.permitted_noninterference`); the reach step is a restriction of the first-order step (`SpacesPlacement.reachStep_iff_admitted`); the delegation core and the transaction layers are not placed	first order

4.5 Evaluation and the common operations

Requirement	Theorems	Instance; class
expression application	β up to terminating administrative firings (`Program.beta_on_step`, `Program.administrative_wellFounded`); an atom applied is stuck (`Program.atom_application_stuck`); the closure machine computes substitution β (`ProgramSubstitution.plotkin_closed`); contextual λ: `LambdaComparison.source_step_iff_executes`	program core; encoding
lexical binding	every halted run returns the uncaptured value (`Program.no_capture_on_step`)	program core; encoding

Requirement	Theorems	Instance; class
correlated against independent choice	shared choice always agrees, independent choices can disagree, over all runs (`Program.shared_vs_independent_on_step`)	program core; encoding
inert quotation, explicit activation	quoted code halts as a value, activation runs it one layer (`Program.quotation_inert`, `Program.activate_quote_on_step`, a run of the machine, not one firing); activation needs code (`Program.activation_needs_code`)	program core; encoding
pattern equations	a call is matched against every equation by two-way unification, one branch per matching equation in program order (`ProgramEquations.answers_exact`); admission is unifiability (`ProgramEquations.call_valid_iff`); the call's own variable is bound by the equation (`ProgramEquations.Controls.variable_bound_on_step`)	ordinary step; encoding
nested calls	call by value, leftmost-innermost, with held data `val` where a position is declared `Atom` (the elaboration by declared type is not formalized); one schema per machine rule, both ways (`ProgramNested.nested_execution`); the next redex is unique (`ProgramNested.decomposition_unique`); reduction in contexts agrees up to renaming of fresh scopes (`ProgramNested.reference_complete`)	ordinary step; encoding
unmatched calls return themselves	one firing with an absence predicate query over the equations of the version read (`ProgramNested.unmatched_fire`, `ProgramNested.u_execution`); under binders, a guarded firing of `(= $C $C)` admitted exactly when no equation matches, final after it fires (`ProgramBinders.returns_iff`, `ProgramBinders.returned_is_final`, `ProgramBinders.evaluates_iff`)	ordinary; contextual guarded
equations under binders	source rewriting by holes with declared parameters is exactly the readings of runs (`ProgramBinders.execution_iff`); principal answers for Miller patterns on the step (`ProgramBinders.principal_answers_on_step`, `ProgramBinders.miller_with_solvers`); controls: η, capture refused, a non-pattern pending at every fuel (`ProgramBinders.Controls.eta_on_step`, `ProgramBinders.Controls.capture_refused_on_step`, `ProgramBinders.Controls.twice_pending_on_step`)	contextual guarded; encoding

Requirement	Theorems	Instance; class
strategies under binders	innermost: one guarded firing per position is exactly one reference step, runs agree both ways for determinate equations, and the checks are decided by certified refutation (`ProgramBinders.Strategy.innermost_step_iff`, `ProgramBinders.Strategy.innermost_execution_iff`, `ProgramBinders.Strategy.innermost_of_certified`, `ProgramBinders.Strategy.innermost_redex_certified`); leftmost-innermost with `Atom` positions: the next redex is unique, and without declarations one step agrees both ways and runs agree for determinate equations (`ProgramBinders.Leftmost.selects_unique`, `ProgramBinders.LeftmostStep.leftmost_step_iff`, `ProgramBinders.LeftmostStep.leftmost_execution_iff`); with declarations sound, and complete when nothing frozen inside the redex matches (`ProgramBinders.LeftmostStep.leftmost_step_sound_atoms`, `ProgramBinders.LeftmostStep.leftmost_step_complete_atoms`)	contextual guarded; complete `Atom` positions on the step *open*, 40%
unused divergent work and call by need	quoted divergent code halts, activated it diverges (`Program.quoted_loop_halts`, `Program.activated_loop_diverges`); call by need at first order: Launchbury's semantics and its machine agree (`ProgramNeed.need_adequacy`), a thunk is evaluated at most once and later demands read its value (`ProgramNeed.later_demand_reads`), agreement with call by value for determinate programs and pure terminating thunks (`ProgramNeed.need_value_agree`), on the ordinary step (`ProgramNeed.need_on_step`); a loop bound and unused answers by need and has no answer by value (`ProgramNeed.need_answers_loop`, `ProgramNeed.value_diverges_loop`); a shared choice is never mixed (`ProgramNeed.choice_never_mixed`)	program core; ordinary step; under binders *open*, 45%
effects	immediate effects of a failed branch stay, deferred ones vanish (`Program.immediate_effects_leave_a_trace`, `Program.deferred_failed_branch_leaves_no_trace`); a deferred block is all or nothing (`Program.deferred_block`) and one firing at the cut (`Program.deferred_one_firing`); `defer` as a source construct (`ProgramSourceDefer.defer_one_firing`)	program core; encoding with computed rules
atomic blocks	a rule family with a uniform bound on made rows cannot commit a larger block in one firing (`TransformQualification.no_bounded_family_commits`, `Program.no_fixed_atomic_commit`)	first order; *proved obstruction* to uniformly bounded rules
suspension	a suspended continuation resumes exactly once per demand (`Program.exactly_once_per_demand`); a read demand replays without bound (`Program.replay_unbounded`)	program core; encoding

Requirement	Theorems	Instance; class
changing definitions	a conclusion does not carry to a new version without a recheck (`Program.unconditional_transport_unsound`); what stays valid (`Program.what_stays_valid`); for pattern equations, adding equations keeps the old answers and appends the new (`ProgramEquations.Versions.answers_append`); an unmatched answer cites the version it read, and installing a later one changes no earlier answer (`ProgramNested.UnmatchedControls.no_retroactive_change_on_step`)	ordinary step; on the binder core *open*, 45%
interpreting programs	a fixed rule family cannot answer every call with variables in one firing (`ProgramCompiled.no_fixed_one_firing_interpreter`); most general unifiers are unique up to renaming (`ProgramMGURenaming.mgu_unique_up_to_renaming`); the fixed interpreter and the compiled program give the same answers up to renaming, both ways, on the step (`ProgramInterpreterJoin.interpreter_join_iff`, `ProgramInterpreterJoin.interpreter_join_on_step`, `ProgramInterpreterJoin.interpreter_join_converse_on_step`)	ordinary step; encoding
communication	rho source paths are exactly firing paths (`ContextualCommunicationAdequacy.source_path_iff_firings`), on configurations whose equations the identity solves, so with no open constraints	occurrence core over the process-term action; encoding
querying	complete answers are exactly the matches at a solution (`ContextualQueryAnswers.actual_answer_iff`); query rows realized by census firings (`CensusSourceCompilation.realize_query_row`)	first order; restriction

4.6 Reflection

Requirement	Theorems	Instance; class
declared syntax and value readings	ordinary queries depend only on represented rows (`Program.semantic_admission_iff`); modulo equivalence is a bisimulation of the ordinary step (`Observation.ModuloEquivalent.semantic_bisimilar`)	ordinary step; restriction
counterexamples to identification	raw queries collide (`Semantic.SemanticControls.syntactic_collision`, `Program.rawQuoted_collision`); the census controls read syntax (`Semantic.persistent_not_semantic`); telling a structural equality of distinct constants is inconsistent (`MetaMeTTaComparison.OperationalEquality.telling_that_structural_equality_would_be_inconsistent`); an admitted object theory makes syntactic refutation unsound and head readouts ill defined (`BetaEtaReadings.Application.refutation_not_admitted`, `BetaEtaReadings.StoredRedex.head_readout_not_invariant`); reduction does not identify quoted syntax (`QuotedLambdaRetained.Controls.reduction_does_not_identify_the_quoted_syntax`)	first order; separation

Requirement	Theorems	Instance; class
preservation under the restrictions	the pure fragment agrees with the original execution (`QuotedLambdaRetained.pure_fragment_agrees_with_original_execution`); quoted code ignores the environment (`RhoContinuation.quoted_code_ignores_environment`); corrected nested reflection is sound and the naive step misses it (`Mettapedia.Languages.ProcessCalculi.MeTTaCalculus.Corrected.Controls.corrected_nested_sound`, `Mettapedia.Languages.ProcessCalculi.MeTTaCalculus.Corrected.Controls.naive_step_misses_nested_reflection`)	contextual; encoding
reflection as a source construct	`reflect` asks a raw syntax question, and its firing is not one of the ordinary step: two configurations with the same rows answer it differently (`ProgramSourceReflection.reflect_not_semantic`, `ProgramSourceReflection.reflect_collision`)	ordinary step extended; separation
absence is an ordinary guard	the unmatched-call query reads through solutions (`ProgramNested.absentQuery_semantic`); on the binder core absence is asked at every future solution (`ProgramBinders.guarded_checks_iff`), and a pending problem is told apart from a refutation by solver results, not by reflection (`ProgramBinders.Controls.twice_pending_on_step`)	ordinary; contextual guarded

4.7 Inference

Requirement	Theorems	Instance; class
valid derivations succeed	reachability of a judgment is exactly a derivation of the independently specified checker (`InferenceKernelReachability.reaches_iff_derivation`); runs reproduce the exact proof (`PersistentInference.realizes_derivation`); published judgments have source derivations (`InferenceSchemaKernel.System.published_judgment_has_source_derivation`); MM0 (`Mettapedia.Logic.TierZero.Adequacy.MM0LocatedReachability.located_kernel_reaches_iff_derives`); typing runs are derivations (`TypingChecking.Kernel.checking_run_sound`, `TypingCheckingMeta.checking_run_sound`)	authority restriction of the first-order step; typing steps
dependent checking as a run	checking tells its obligations into the store, a solution of the store yields the derivation, and an unsolvable store refuses (`TypingCheckingMeta.checking_run_sound_of_store`, `TypingCheckingMeta.refused_of_unsolvable`); the checking step works at a fixed metavariable frame and allocates no fresh metavariable, and its completeness holds when the goal is typed at the identity (`TypingCheckingMeta.switch_complete`, `TypingCheckingMeta.app_complete`)	typing step

Requirement	Theorems	Instance; class
the certain checks as a calculus	a derivation calculus for the first-order checks is sound for free Henkin models and complete at generic points (`AnnotatedRuns.CertainChecks.derives_sound_free`, `AnnotatedRuns.CertainChecks.derives_complete_generic`); the Herbrand model is sound and not complete (`AnnotatedRuns.CertainChecks.oneConstant_separates`); a query-free firing is admitted exactly when its admission is derivable (`AnnotatedRuns.CertainChecks.admitsDerivably_iff`)	first order
forged evidence fails	a public row cannot serve as a protected premise (`InferenceAuthorityControls.wrong_space_premise_refused`); a counterfeit conclusion is no activation (`InferenceAuthorityControls.fabricated_output_refused`)	first order
wrong premises fail	one row cannot supply distinct judgments (`InferenceProjectionControls.one_row_cannot_supply_distinct_judgments`); a ground term is not a function (`TypingCheckingControls.ground_not_function`)	first order; typing
capture fails	assignments cannot capture bound indices (`ScopedInferenceValueControls.contextual_assignment_cannot_capture_its_bound_index`); resolution refuses capture (`Resolution.Controls.capture_refused`)	first order
missing authority fails	a non-writer creates no protected occurrence (`SchemaPublicationAuthority.System.nonwriter_creates_no_protected_occurrence`); a client cannot publish (`InferenceAuthorityControls.client_cannot_publish`)	authority restriction
execution confers no validity	the forged producer is admitted by the step and published by no step of its process (`TransformQualification.forged_admitted_but_not_published`); runtime acceptance of a void contract the step refuses (`TypingContractsControls.void_trap`); symbolic admission of a hole of an empty type (`Interface.Dependent.trap_helper`)	*necessary extension*: authority and closed asks
guards by polarity	"never bad" has a finite certificate exactly when it holds and the datum has finitely many reachable states (`GuardFragments.exists_invariant_iff`); the budgeted closure verdict certifies exactly on finite-state data and is suspended at every budget on infinite-state data (`GuardFragments.always_iff_exists_commit`, `GuardFragments.suspended_of_infinite`); on open data a reachability guard is decided at a principal answer and a "never" guard is not stable (`GuardFragmentsOpen.eventually_entails_iff`, `GuardFragmentsOpen.Controls.always_not_stable`)	interface asks; restriction

Requirement	Theorems	Instance; class
lowering by checked certificates	where an invariant kept by every admitted rule makes the specification equivalent to the existence of a certificate, the runs with the specification are exactly the runs with the check, certificates erased (`GuardLowering.runs_spec_iff_lowered`, `GuardLowering.certified_runs`); a checked certificate is sound whoever made it (`GuardLowering.admitted_of_certificate`); a fragment not closed under successors admits wrongly (`GuardLowering.Controls.closure_free_admits_wrongly`); a guard priced by the store it scans breaks interleaving invariance (`MeteredGuards.Controls.scan_price_order`)	first order; encoding
fair search makes certificates	a monotone search firing, enabled and weakly fair, fires or is preempted (`FairSearch.fires_or_preempted`); on finite-state data every weakly fair run's store becomes a certificate the lowered guard checks (`FairSearch.fair_search_certifies`); in one store the answer is sound under every schedule and arrives under weak fairness (`FairSearchGuards.answer_sound`, `FairSearchGuards.answer_eventually`); search firings that take are not complete under fairness (Example 8)	commits; restriction to monotone search

5 Minimized examples

Each example is the smallest we know of on the common structure. All are kernel-checked.

Example 1 (Open unification; `TransformQualification.Examples.OpenUnification`). Rows $0 : p(1)$ and $1 : p(2)$ in the first-order step. After reading row 0 through $p(X_0)$, reading row 1 through the same caller variable is refused, and through a fresh variable is admitted (`shared_second_read_refused`, `fresh_second_read_admitted`). The tells $X_0 \doteq 1$ and $X_0 \doteq 2$ are each admitted and never both (`tells_conflict`), and the two failed compositions leave prefixes that every solution distinguishes (`prefixes_differ`); a compatible pair runs (`compatible_run`). Binder crossing is not first order; its minimal pair is the pattern equation $m(x) \doteq n(x)$, principal with η and not without (`ContextualDependentFibre.Controls.point_pattern_principal`, `ContextualDependentFibre.Controls.point_pattern_no_principal`).

Example 2 (Absence; `TransformQualification.AbsenceExample`). Row 0 is *cancel*(0), rows 1, 2 are requests. The firing *ask* takes request 1 under the guard that no *cancel*(u) is present. With u unknown it is refused; after the tell $u \doteq 1$ it is admitted; after $u \doteq 0$ it is refused. After $u \doteq 1$, inserting *cancel*(1) refuses it and inserting *cancel*(2) keeps it (`insertions_separate`). Moving the guard into an empty-census predicate query gives the same verdicts (`ask_as_query_refused_unknown`, `ask_as_query_admitted_after_one`).

Example 3 (Composition). At the empty state, *tell*($X \doteq 1$) and *ask*($X \doteq 1$) have trace-independent footprints; only the order tell-then-ask runs (`AnnotatedRuns.Controls.told_check_one_order`). Three tells $X \doteq Y$, $X \doteq 1$, $Y \doteq 2$ are pairwise independent and form no cell (`AnnotatedRuns.CellControls.hollow_cube`); natively, three pairwise clear tellers have no order at all (`Interface.NativeIteratedControls.pairwise_not_enough`).

Example 4 (Resources; `TransformQualification.Examples.Resources`). Rows $0 : c$ and $1 : c$. Taking c twice takes both (`take_two_admitted`); taking 0 twice is refused (`take_same_twice_`

refused); reading 0 twice is admitted (aliased_read_admitted); read-then-take runs, take-then-read is refused (read_then_take, read_after_take_refused).

Example 5 (Evaluation and reflection). On the program core: one shared choice used twice agrees with itself in every halted run, two independent choices disagree in some (Program.shared_vs_independent_on_step). Quoted Ω halts and activated Ω diverges (Program.quoted_loop_halts, Program.activated_loop_diverges). Two rows $20 : X$ and $21 : Y$ under $X \doteq Y$ are equivalent modulo the knowledge, and a query for the label Y itself separates them, while the query read through solutions does not (Semantic.SemanticControls.syntactic_collision, Semantic.SemanticControls.noYSemantic_agrees).

Example 6 (An unmatched call returns itself; ProgramNested.UnmatchedControls). The program is

```
(= (g a) b)    (= (f $x) (out $x))
```

and the call is `(f (g c))`. No equation matches `(g c)`, so its call returns the data `(g c)` by one firing whose absence predicate query sees no matching equation at its cut; the outer call then matches, and the run answers `(out (g c))` (inner_value_outer_call_on_step). Without the return firing the machine has no step at the inner call (ProgramNested.Controls.unmatched_inner_stuck). Versions: under a version with no equation for `f`, `(f a)` answers itself and cites that version (answered_as_itself_v1_on_step); where `(= (f a) b)` is installed the return is refused (unmatched_refused_v2); installing it later changes neither the earlier answer nor its citation (no_retroactive_change_on_step). Under a binder, with only `(= (f (λ x. $N)) $N)`, the call `(f (λ y. y))` matches nothing, since `$N` may not capture `x`, and returns itself (ProgramBinders.UnmatchedControls.capture_returns_itself).

Example 7 (Inhibition: one atom forces a race; Contention.Controls). The space holds one occurrence `a`. Two firings race for it, with positive weights w_r and w_v:

```
racer: take a, make out    vetoed: take a, guard (absent p), make out
```

At this space `racer` wins with probability $w_r/(w_r + w_v) < 1$; after one atom `p` is added from outside, `vetoed` is refused and `racer` wins with probability 1 (forcing_control). Without inhibition no context forces a race over positive weights (Contention.no_context_forces), and no added atoms force `vetoed` (cannot_force_vetoed). The same inhibition separates the two readings of contention: a firing that takes `a` and makes `p`, and one that takes `b` under `(absent p)`, share no occurrence, yet the first refuses the second; normalizing within shared occurrences gives local likelihoods of total mass 2, and normalizing within interference gives the step's own, of total mass 1 (inhibition_control).

Example 8 (A take race that fair search never completes; FairSearch.Controls). The space holds the goal `goal`, and two search firings race for it:

```
take goal, make dead    take goal, make proof
```

A run in which the first wins and then idles is strongly fair for every task, has unbounded budget, and never makes `proof` (race_stronglyFair, race_never_proves), although `proof` is reachable (race_proof_reachable). Reading the goal instead of taking it, every weakly fair run proves (read_race_proves). Adding the repair `take dead, make proof` reaches exactly the same stores (race_reachable_iff), and every weakly fair run of the repaired race proves (repair_fair_proves). So reachability does not decide fair completeness (reachability_does_not_decide_fair_completeness). What separates the two races is that a proof stays reachable from every reachable store, a property bisimilarity preserves and reachability does not (FairSearchIdentity.alwaysPossibly_of_bisimilar, FairSearchIdentity.search_identity_is_branching); that this property with strong fairness gives completeness on every finite-state system is open.

Example 9 (Inference). The identity is typed by a run of the typing step and a ground term against a function type is refused (`TypingCheckingControls.identity_run`, `TypingCheckingControls.ground_not_function`). The producer of a protected occurrence is admitted by the first-order step and reaches its result by no step of the client's process (`TransformQualification.forged_admitted_but_not_published`).

6 Full programs from the core

The interpreter core. A source language over S-expressions with closures, lexical binding, quotation and activation, choice, facts added and removed, suspension and versioned definitions has an independent reference semantics of 24 rules. Each rule is a schema of take and read patterns and made terms, copied into a fresh scope at each firing; the two-way unifier solves the equations. One source step is exactly one firing of the ordinary step, in both directions, for every program (`Program.step_iff_rewrites`, `Program.program_interface_iff`), and the administrative firings terminate (`Program.administrative_wellFounded`). The laws of evaluation in Section 4 are theorems about these runs.

Pattern equations. A program is a list of equations `(= (f p1 ... pn) rhs)`, each compiled to one schema that takes the call and makes the instantiated right-hand side; the step's own knowledge does the unification. Each matching equation gives exactly one branch, in program order, duplicates included, and a call that matches nothing has no step (`ProgramEquations.answers_exact`, `ProgramEquations.stuck_iff`); the reference semantics is an independent inductive relation over a semantic notion of most general unifier (`ProgramEquations.equations_execution`). Calls inside arguments and right-hand sides are evaluated by call by value, leftmost-innermost, the reading of the HE specification, with positions declared `Atom` written as held data (`ProgramNested.nested_execution`). A call that matches nothing returns itself by an absence predicate query, ordinary, exact at its cut and stable under later bindings, and its answer cites the version it read (Example 6).

Compiled and interpreted. A fixed interpreter of 13 equations answers calls with variables by allocating one fresh variable per slot in administrative firings and unifying the renamed copy in one firing; its answers are the compiled ones up to renaming, in both directions, at the source and on the step (`ProgramInterpreterJoin.interpreter_join_iff`, `ProgramInterpreterJoin.interpreter_join_converse_on_step`). The allocation is forced: a fixed rule family cannot answer every call with variables in one firing (`ProgramCompiled.no_fixed_one_firing_interpreter`). This is the positive reflective result: an interpreter on the step that runs stored equations as data.

Equations under binders. On the contextual guarded step, over any binding signature, an equation is a pair of terms with holes that list the object variables their solutions may use. One schema per equation; the equation `lhs` $\doteq$ `call` joins the retained store, and a hole no equation constrains stays unknown, to be bound by a later firing. Source rewriting is exactly the readings of runs (`ProgramBinders.execution_iff`), and for Miller patterns the actual worklist computes one principal answer at every firing (`ProgramBinders.principal_answers_on_step`, `ProgramBinders.miller_with_solvers`). The λ calculus and the quoted-λ account are instances: β is `(= ((λ x. $B x) $A) $B[$A])` and activation is `(= (activate (quote $C)) $C)` (`ProgramBinders.Nested.lambda_three_way`, `ProgramBinders.Quoted.executes_iff_rewrites`). Absence of a matching equation is a guard per equation, decided by certified refutation, and it gives unmatched calls that return themselves and innermost and leftmost-innermost reduction under binders (Section 4). On embedded first-order rows the binder core agrees with the first-order core (`ProgramBinders.FirstOrder.step_join`, `ProgramHosts.absence_agrees`).

Call by need. At first order, a heap cell holds a thunk; the first demand marks it busy, evaluates it and writes the value, and every later demand reads the value. Launchbury's semantics, its machine and the ordinary step agree (`ProgramNeed.need_adequacy`, `ProgramNeed.need_eval_on_step`); each cell is its own row, and a demand of a value reads the row without taking it.

Computed rules. A rule family with a uniform bound on made rows bounds what one firing commits (`TransformQualification.added_card_le`, `TransformQualification.no_bounded_family_commits`). Atomic blocks of unbounded size are therefore realized by a rule computed per block, which is still one firing of the step at the block's cut (`Program.deferred_one_firing`, `ProgramSourceDefer.defer_one_firing`, `StagedSourceEffects.refused_has_no_residue`), or by a compound occurrence carrying the block.

Compiled clauses and contextual executions. Compiled source clauses are admitted exactly as native firings with their resource and original-cut queries (`TransformQualification.Native.admitted_iff_native`). Contextual guarded firings, which carry the λ and quoted-λ executions, are admitted exactly as query-free native firings (`TransformQualification.Native.guarded_valid_iff_native`).

Inference hosts. Steps of schema processes are exactly authorized first-order firings (`TransformQualification.schema_step_iff_admitted`); validity lives in the authority restriction and in the derivation adequacy of the host, never in admission alone.

What programs still lack. The two program hosts are joined by agreement, not by one program core over the interface: a single core needs frames that can adjoin a schema's holes, an operation `Interface.Frames` does not have (55%). Programs at dependent frames are not built (45%). On the binder core: versions, which need a state of two occurrences with absence scoped to the call (45%); complete `Atom` positions on the step, since whether a position is frozen depends on heads a solution may still choose (40%); call by need (45%); runs from the reference to the step for non-determinate programs (40%). Call by need with open data and two-way unification (55%); answers up to η in matching (50%); fairness and cost.

Necessary extensions. A rule generator for atomic blocks; the authority layer for publication; closed asks for checked holes; a born ledger for identities (`Interface.nativeUnborn`); an η frame category; disequation frames for disequality tells. Each is placed above or inside the same step.

7 Recommendation

7.1 Retain

Retain the rule of Definition 2 as the operational core of the calculus: one firing that takes, reads and makes occurrences, tells equations, and asks its guards and predicate queries at every solution of its own information, over an abstract category of frames. Justification: every proved row of Section 4 is a theorem about an instance of it; the admission characterization is the same theorem in every instance; each feature holds at the interface or on a named restriction with a control and a bridge, or is a named gap (Section 3); the first-order and second-order steps are the dependent step's fibre wherever the unifier decides, and that condition is inhabited on both sides; no obstruction to the rule is known.

7.2 Revise

Revision	Forcing theorem
transport up to dead names, with a partial move	`Interface.NativeUpToDeadLaws.strict_separation_fails`, `Interface.NativeResidualSeparation.residual_not_separated`; partiality: `Interface.NativeDead.Controls.deadQuery_realized` with `Interface.NativeDead.Controls.carrying_not_surjective`
helper firings agree up to the exchange of their telescopes	`Interface.DependentFull.TelescopeControls.composites_differ`, `Interface.DependentFull.SwapControls.composites_differ`
waves need joint solvability	`Interface.NativeIteratedControls.pairwise_not_enough`, `Interface.NativeIterated.no_order_of_not_jointly`, `AnnotatedRuns.CellControls.hollow_cube`
principal pattern answers need η	`Interface.Translation.pattern_no_principal`, `ContextualDependentFibre.Controls.point_pattern_no_principal`
ordinary and reflection queries split	`Semantic.SemanticControls.syntactic_collision`, `Program.rawQuoted_collision`, `ProgramSourceReflection.reflect_collision`
atomic blocks need computed rules or a compound occurrence	`TransformQualification.no_bounded_family_commits`, `Program.no_fixed_atomic_commit`
an interpreter allocates fresh variables in administrative firings	`ProgramCompiled.no_fixed_one_firing_interpreter`
freshness needs the born ledger	`Interface.NativeFreshnessControls.maker_not_realized`, `Interface.native_run_protocol_iff`
consistency and inhabitation are two asks	`Interface.Dependent.trap_helper` (relative to an empty type), `ContextualDependentClosedBoundary.Controls.point_not_closedSolvable` with `ContextualDependentClosedBoundary.Controls.ground_uninhabited`
publication is an authority restriction	`TransformQualification.forged_admitted_but_not_published`
reusable outputs are snapshots	`Interface.NativeSnapshot.snapshot_obstruction`; they commute under conservativity (`Interface.NativeSnapshot.snapshots_commute`)
open systems need labels for the context	`Observation.CongruenceControls.closed_not_congruence`
disequality is a frame extension, told only on restrictions	`DisequalityFrames.Restriction.guess_breaks_frame`; principal answers in constrained frames: `Disequality.Controls.no_presenting_answer`
independence across spaces is a restriction, since the knowledge is shared	`Spaces.Controls.shared_variable_crosses_spaces`
rights alone do not bound writes; a ticket per write does	`Permissions.Controls.rights_alone_unbounded`, `Permissions.made_le_tickets`
contention includes inhibition	`Contention.Controls.inhibition_control`, `Contention.Controls.forcing_control`
a “never” guard is certified by a checked certificate, which exists exactly on finite-state data	`GuardFragments.exists_invariant_iff`, `GuardFragments.suspended_of_infinite`, `GuardFragmentsOpen.Controls.always_not_stable`
reachability does not decide fair completeness; the identity of searches is branching	`FairSearch.Controls.reachability_does_not_decide_fair_completeness`, `FairSearchIdentity.search_identity_is_branching`
the dependent unifier needs more rules than the earlier set	`ContextualDependentFragment.Controls.fragment_not_decisive`, `ContextualDependentFragment.Controls.blocked_not_decisive`

7.3 Reject

Each hypothesis below is rejected by an obstruction under the stated adequacy criterion.

1. *Native diamonds close up to isomorphisms of states.* Criterion: residual solutions agreeing along both carrying morphisms agree. Obstruction: `Interface.NativeResidualSeparation.residual_not_separated`.
2. *Transport up to dead names is total on firings.* Criterion: every admitted firing at the larger state is carried from the smaller. Obstruction: `Interface.NativeDead.Controls.deadQuery_realized`, `Interface.NativeDead.Controls.carrying_not_surjective`.
3. *Pairwise independence suffices for a wave.* Criterion: some order of the family runs. Obstruction: `Interface.NativeIteratedControls.pairwise_not_enough`.
4. *Second-order frames without η, raised into dependent frames, keep principal pattern answers.* Criterion: a principal answer in the dependent frames, over a package with no root computation and symmetric head equality. Obstruction: `Interface.Translation.pattern_no_principal`.
5. *Raw label queries are ordinary.* Criterion: configurations equivalent modulo the knowledge admit renamed firings together. Obstruction: `Semantic.SemanticControls.syntactic_collision`.
6. *A rule family with a uniform bound on made rows commits every atomic block in one firing.* Criterion: one block, one firing adding the block's rows. Obstruction: `TransformQualification.no_bounded_family_commits`. A family with no uniform bound, such as one rule per block size, and a block packaged as one compound occurrence are not excluded.
7. *Absence from the space is freshness.* Criterion: the commit protocol realizes the run. Obstruction: `Interface.NativeFreshnessControls.maker_not_realized`.
8. *Symbolic admission certifies a checked hole.* Criterion: a closed solution exists. Obstruction: `ContextualDependentClosedBoundary.Controls.point_not_closedSolvable`.
9. *Admission confers publication of a judgment.* Criterion: the process's step reaches the result. Obstruction: `TransformQualification.forged_admitted_but_not_published`.
10. *The literal prototype step simulates the published register semantics with identical outputs.* Obstruction: `Mettapedia.Languages.MeTTa.OSLFCore.MetaMeTTa.Controls.no_identity_output_simulation`; a different encoding is not excluded.
11. *The earlier rule set of the dependent unifier decides the dynamic pattern fragment.* Criterion: every fragment store is solved principally or refuted at some budget. Obstruction: `ContextualDependentFragment.Controls.fragment_not_decisive`, `ContextualDependentFragment.Controls.blocked_not_decisive`. The question for the current solver is open.
12. *A fixed rule family interprets every call with variables in one firing.* Criterion: one firing makes the answer row. Obstruction: `ProgramCompiled.no_fixed_one_firing_interpreter`.
13. *Firings at different spaces are independent.* Criterion: a firing at one space leaves admission at another unchanged. Obstruction: `Spaces.Controls.shared_variable_crosses_spaces`.
14. *A rights table alone bounds what a space receives.* Criterion: a bound on the rows made by permitted runs. Obstruction: `Permissions.Controls.rights_alone_unbounded`.
15. *Contention is the sharing of an occurrence.* Criterion: local normalization gives the step's probabilities, of total mass 1. Obstruction: `Contention.Controls.inhibition_control`.
16. *No context forces a race over positive weights,* with predicate queries. Criterion: added atoms leave every competitor below probability 1. Obstruction: `Contention.Controls.forcing_control`; without inhibition the hypothesis holds (`Contention.no_context_forces`).
17. *Reachability of proofs decides completeness of fair search.* Criterion: two searches with the same reachable stores are both fairly complete or both not. Obstruction: `FairSearch.Controls.reachability_does_not_decide_fair_completeness`.
18. *A guard may be priced by the store it scans.* Criterion: the price of a history is invariant under

its interleavings. Obstruction: `MeteredGuards.Controls.scan_price_order`.

7.4 What stays open

Open row	Est.	What is missing, and the effect on the verdict
decisiveness of the dependent unifier on the whole dynamic pattern fragment	45%	progress of the pattern, in-place and prune rules on every Miller-pattern equation (40%); the three unread rigid shapes, by one reader of elimination spines and spine inversion through η (60%, 45%); closure of the stratified class under answers (45%). Proved: solver completeness on the fragment, and through the fibre on translated pattern stores. Refuted for the current solver: more rules are needed, as after the refutation of the earlier set, or the fragment is narrowed. The rule is unaffected, since residuals stay pending without loss
pruning under an older metavariable	50%	base-type instantiations keep the normal form of the instantiated term; outside the fragment
census queries and multi-frame queries through the join	60%, 65%	a meaning for arbitrary regions at dependent solutions; absence and counting through the multi-frame instance
the endomorphism condition for the first-order instance	45%	endomorphic answers need inhabitants of the dropped metavariables' types, which can be empty
principality against arbitrary dependent telescopes	40%	second-order principal answers against solutions outside the image; admission does not need it
step bisimilarity for helper firings; the dead-helper converse	60%, 45%	a strict transport; strengthening of typed judgments and query tests natural under weakening
located labels at dependent frames	70%	query regions that respect spaces, removing the remaining hypotheses of the dependent spaces theorem
guarded native firings with disequations; a label law for native rows; the four-way family on the native step	55%, 60%, 65%	checks asked at separating solutions only; term labels through publication
one program core over the interface; programs at dependent frames	55%, 45%	frames with schema extensions
the binder program core: versions, complete `Atom` positions, call by need, non-determinate runs	45%, 40%, 45%, 40%	see Section 6; no effect on retain or reject
results of contextual guarded executions against native published states	70%	admission is placed; results are not. Would not change the verdict
native transport for recipes, reusable outputs and closure queries	50%	refines the revisions (patterns as queries; conservative reuse); a closure query does not determine its residual (`Interface.NativeClosingControls.closure_extensions_disagree`)
full abstraction and congruence for contextual and native firings	50%	would strengthen observation; no effect on retain or reject
causal Bayesian networks on the step	70%	the confusion-free race identity for cells enabled by joins, then the staged network program (65% after it) and replay (60%)
"never" guards on infinite-state data; fairness in general	60%, 70%	search firings discharging a symbolic closure obligation; strong fairness with "a proof stays reachable" giving completeness on finite-state systems
grants as firings	70%	delegation, the sealed case and revocation; the permission vocabulary is unsettled

Open row	Est.	What is missing, and the effect on the verdict
a checker generated from the calculus's own rules	50%	`checkProof = accept` exactly for derivations, generated from admitted rules; located kernel adequacy covers MM0 only
behavioural modalities over the rule	60%	$\Diamond$ and $\Box$ of the OSLF framework over `Interface.Step`; the one link is that observed bisimilarity preserves "a proof stays reachable" (`FairSearchIdentity.alwaysPossibly_of_bisimilar`)
HE semantics and a uniform register-semantics encoding	50%	comparisons; no effect on the rule

8 Addendum: results after the revision

Two results arrived after the revision above. They change no row of the recommendation; they narrow two open rows.

8.1 Firing of the pattern rules

The open-unification row of Section 4 named firing on Miller-pattern equations as the main gap. The pattern rules now provably make progress, at a large enough budget, on three rows of the census:

- **inversion** for the newest metavariable, $F\,\bar{x} \doteq t$ with t over $\bar{x}$ and older declarations (`ContextualDependentPatternFiring.firingRound_fires_inversion`);
- **inversion in place**, for an older metavariable solved past the later declarations (`ContextualDependentPatternFiring.placedRound_fires`);
- **refutation by a rigid local outside** $\bar{x}$, even with flexible spines inside t (`ContextualDependentPatternFiring.firingRound_fires`, `ContextualDependentPatternFiring.refuted_rigidOccurrence`).

Their hypotheses are inhabited: each is instantiated on a tower store (`ContextualDependentPatternFiring.Controls.inversion_fires`, `ContextualDependentPatternFiring.Controls.placed_fires`, `ContextualDependentPatternFiring.Controls.rigid_fires`). One reader of elimination spines, along the literal Π/Σ types of the prefixes, reads the nested and applied projections (`ContextualDependentEliminations.elim_exact`, `ContextualDependentEliminations.elim_weight`), and the stratified class is closed under an answer for the newest metavariable (`ContextualDependentStratifiedClosure.stratified_instantiate_newest`). Each new layer acts only where the one below it is stuck, so every earlier decision is kept (`ContextualDependentEliminations.elims_decided_of_placed`).

Still open. Four flexible rows (intersection, pruning under rigid structure, flex–flex with a declaration between, and the cumulative argument) and the spine at a flexible base type are in the fragment, solvable, and stuck at budget 16 (`ContextualDependentPatternFiring.Controls.unread_flexible_rows`, `ContextualDependentEliminations.Controls.flexibleBase_elims_suspended`). The four flexible rows need a rank under which a step that keeps every declaration still decreases; closure under in-place answers is open. Decisiveness on the fragment stays open.

8.2 Causal networks on the step

The composition row now carries Pearl's calculus. When every race, once open, persists until it fires, normalizing each choice within its own race gives exactly the step's trace probabilities

summed over orders, also through joins (`CausalCells.CellSystem.History.race_identity`). A causal Bayesian network is then a program: each variable's schema takes `(pend v)`, reads `(reg v r)` and its parents' `(val p b)`, and makes `(val v b)`, weighted by its table; its likelihood is the network's joint (`CausalNetworksStep.step_likelihood_eq_jointWeight`). A restriction at the variable's handle gives the truncated factorization under positivity (`CausalNetworksStep.step_do_likelihood`); telling a forcing regime atom gives it always, and equals conditioning on the regime (`Mettapedia.GSLT.Causality.StepBridge.step_regime_likelihood`, `Mettapedia.GSLT.Causality.StepBridge.regime_is_conditioning`, `Mettapedia.GSLT.Causality.StepBridge.do_is_regime`). Replay of recorded draws is the counterfactual (`CausalReplay.replay_likelihood`, `CausalReplay.replay_average`), and the network's graph is read off the step's independence (`CausalSeparation.history_staticIndependent_iff`).

8.3 The C runtime and proof-system integration evidence

The standalone `tcalc` runtime executes rule tables used by Micromath, MM0/MMB, HOL Light proof replay, Megalodon and Dedukti implementations. Micromath's differential operation suite is not a `set.mm` verification. HOLR records are primitive proof nodes in a local format; their counts are not counts of distinct exported mathematical theorems.

The Megalodon implementation checks explicit natural-deduction proof terms, with additional typing and signature rules. The Dedukti implementation has explicit LF/CoC and beta/eta profiles. Named C conversion, substitution and context helpers remain in their trusted bases. Reference/corpus agreement and mutated-proof rejection are regression evidence, not general adequacy or conservativity theorems. Different source-elaboration and proof-replay timings are not matched-input kernel speed comparisons.

The prepared-effect/admission/event C bridge follows the mathematical layering. Its source-state guard evidence and cursor-resumption controls are relevant realization tests, but the complete C-to-Lean correspondence and the live CeTTa cursor integration remain open. Runtime permission records must preserve the original event and observation contract; a new profile cannot manufacture missing evidence.

The exact 211-byte `flat_fire.S` reference function is proved by `FLAT_FIRE_CORRECT` in HOL Light over s2n-bignum's x86 model, with no new axioms. The theorem's memory/ABI assumptions include scratch separation, valid modular 32-bit offsets and the 56-byte stack/return exclusion. The pointer-based runtime packing adapter is not covered by that theorem.

The Lean declarations cited in this addendum were audited separately: all use `propext`, `Classical.choice` and `Quot.sound`, or fewer, and none uses `sorryAx`.

9 Receipts

The recorded Lean audit covers the metatheory declarations; the C implementation and HOL machine-code theorem above have their separate evidence boundaries. Unqualified Lean names in an example are in the namespace in its title, and those in the instance columns of the feature table are in `InterfaceTable`. One audit file imports 229 modules that own the cited declarations, with warnings as errors, and prints the axioms of all 492 of them: 429 use `propext`, `Classical.choice` and `Quot.sound`, 42 use `propext` and `Quot.sound`, 5 use `propext` alone, and 16 use no axiom; none uses `sorryAx`, and every name resolves to a declaration of the `Mettapedia` tree.

Vocabulary

frame an object of the frame category: a declaration family, a formed telescope, or the one object of a substitution action;

take, read, make what a firing removes from the space, observes there without removing, and adds to it with fresh identities;

tell an equation a firing adds to its information;
guard an ask of a firing at every solution of its information;
knowledge the equation store of a state; **information**: that of a firing at its target;
space the present occurrences of a state, or those at one **handle**;
rights what a firing may do at a handle: `receive` (take), `peek` (read), `send` (make), `observe` (query);
schema a rule whose variables are opened in a fresh scope at each firing; its **capture set** lists the caller variables it shares;
birth/guard cut the information at which a firing's guards are asked;
predicate query a complete census of a region at every solution;
dead names names of a larger frame that no row, knowledge or ledger entry mentions;
authority restriction the firings a process may fire, by publication rights;
fibre a restriction of the frames with stronger guarantees.